Development and testing of an Erlang-based research management system: A client-server approach

Flavia Foca

Laurea Magistrale in Informatica e Innovazione Digitale
Università degli Studi di Urbino Carlo Bo

f.foca@campus.uniurb.it

October 22, 2026

1 Introduction

1.1 Application context

Research activities often involve multiple researchers and tasks that need to be organized and associated with one another.

Even in a simplified setting, managing these elements requires maintaining information about individual projects, identifying the researchers involved, and keeping track of their assigned activities.

The aim of this project was to develop a small research management system capable of representing these relationships and supporting basic operations on the associated data.

The proposed application is intended as a simplified representation of research project management rather than a complete solution for use in a real research organization.

1.2 Motivation for using erlang

Erlang[1] was selected as the programming language for this project primarily to meet its educational objectives, providing an opportunity to apply concepts such as concurrency, message passing, pattern matching, persistent storage, and distributed programming within a concrete application.

In particular, Erlang's process-oriented programming model supports concurrency by allowing independent processes to communicate through asynchronous message passing. This characteristic was suitable for implementing a client-server architecture in which multiple clients can submit requests to a central server process responsible for managing shared data.

Erlang also supports communication between processes running on different nodes, making it possible to explore distributed programming through a configuration involving a server node and two client nodes.

Another relevant feature is pattern matching, which was used to distinguish incoming requests according to their structure and to interpret the results of database queries.

2 System architecture and implementation methodology

2.1 Data model and persistence

The data model was implemented using Erlang's Disk-Based Term Storage (DETS), which stores Erlang terms on disk rather than in memory, as in the case of Erlang Term Storage (ETS). This approach allows data to be retained across application restarts. This feature was considered suitable for the proposed research project management system, where project-related information must remain available beyond the lifetime of individual server processes. The database consists of four DETS tables, each responsible for a specific component of the application. The `progetti` and `ricercatori` tables store information about research projects and researchers, respectively. The `tasks` table stores the activities associated with research projects, including their assignment and contribution to the overall project workload. Finally, the `partecipazioni` table represents the associations between researchers and projects, allowing multiple researchers to participate in the same project and each researcher to be associated with multiple projects. Three tables `progetti, ricercatori, tasks` were defined using the set type, which ensures that each key uniquely identifies a stored record. In contrast, the `partecipazioni` table was defined as a bag, allowing multiple records to share the same project identifier while preventing the insertion of identical tuples. This distinction supports the representation of project memberships without requiring a separate table for each project. The structure and purpose of the four DETS tables are summarized in Table 1.

Table 1: Structure and purpose of the DETS tables.

DETS table	Type	Record structure	Description
`progetti`	`set`	`{Id, Nome, Tipo}`	Stores project identifiers, names, and types.
`ricercatori`	`set`	`{Id, Nome, Ruolo}`	Stores researcher identifiers, names, and roles.
`tasks`	`set`	`{TaskId, ProjectId, Descrizione, ResearcherId, Stato, Percentuale}`	Stores project-related tasks, including the assigned researcher, status, and percentage contribution.
`partecipazioni`	`bag`	`{ProjectId, ResearcherId}`	Associates researchers with research projects.

2.2 System architecture

The logical architecture of the proposed system is illustrated in Figure 1. The application follows a centralized client-server model in which client processes communicate with a central Erlang server process through message passing. The server is responsible for processing incoming requests and accessing persistent data stored in four DETS tables. This separation ensures that data management operations are handled centrally, while clients interact with the system through a request-response mechanism.

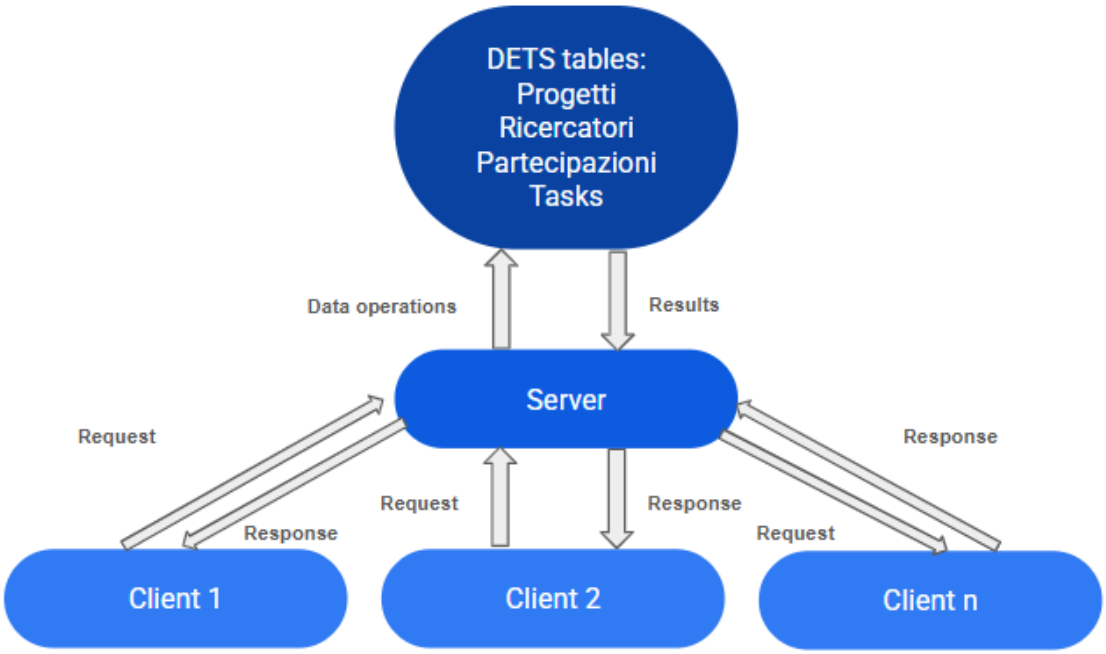


Figure 1: Logical architecture of the research project management system.

2.3 Server process and lifecycle

The lifecycle of the central server process is illustrated in Figure 2.

The central server is started through the `start/0` function, which creates a dedicated Erlang process using the `spawn/1` primitive.

The newly created process executes the `init/0` function, which opens the four DETS tables required for persistent data management. Once initialization is completed, the server enters a recursive message-processing loop implemented through the `loop/0` function.

The server continuously waits for incoming requests using Erlang's `receive` construct. Each received message is matched against a predefined set of patterns corresponding to the supported operations, such as project creation, researcher registration, task insertion, and project retrieval.

Once a request has been processed, the server sends the corresponding result to the requesting client and recursively invokes `loop/0` to handle subsequent messages.

The use of tail recursion allows the server to maintain a persistent request-processing cycle without accumulating additional stack frames.

The server also supports a controlled shutdown procedure. Upon receiving a `stop` message, it closes the open DETS tables and terminates its execution.

2.4 Pattern matching and message dispatch

Pattern matching plays an important role in the implementation of the Erlang-based research management system. It is used to identify incoming requests, extract their parameters, and interpret the results of database operations.

In particular, the server uses pattern matching within its `receive` expression to distinguish different operations according to the structure of incoming messages. For example, a message matching the pattern `{add_researcher, Id, Nome, Ruolo, From}` triggers researcher insertion, whereas a message matching `{get_researcher, Id, From}` triggers researcher retrieval.

Pattern matching is also employed during database validation. For example, the `add_researcher/3` function uses a `case` expression to interpret the result of a DETS lookup, as illustrated in Listing 1.

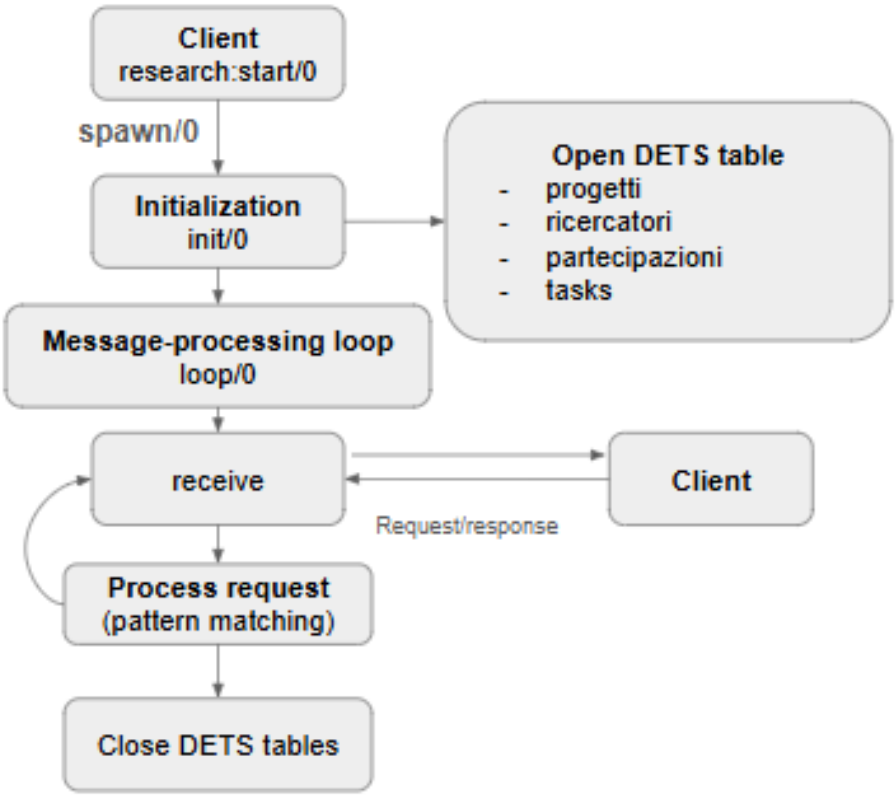


Figure 2: Lifecycle of the central Erlang server process.

Listing 1: Example of pattern matching in database validation.

```
case dets:lookup(ricercatori, Id) of
    [] ->
        dets:insert(ricercatori, {Id, Nome, Ruolo});
    [_] ->
        {error, researcher_already_exists}
end.
```

The expression distinguishes two possible outcomes. An empty list (`[]`) indicates that no researcher with the specified identifier exists, allowing the new record to be inserted. Conversely, a single-element list (`[_]`) indicates that the identifier is already present, causing the function to return an error tuple.

Finally, pattern matching is also used by client processes to extract the results returned by the server through messages of the form `{result, Risultato}`.

2.5 Request handling and data operations

The system provides a set of functions for inserting, retrieving, and validating data stored in the DETS tables. These operations are implemented through dedicated Erlang functions, each responsible for a specific aspect of research project management.

Data insertion is supported through the `add_project/3`, `add_researcher/3`, `add_participation/2`, and `add_task/6` functions. These functions use `dets:insert/2` to store records in the corresponding DETS tables.

Project retrieval is implemented through the `get_project/1` function, which uses `dets:lookup/2` to search for a project based on its identifier. The operation returns the matching record, if present, or an empty list if no corresponding project is found.

To prevent duplicate project identifiers, the `add_project/3` function performs a preliminary

lookup before inserting a new record. If the identifier already exists, the function returns an error tuple instead of inserting the project.

An additional validation mechanism is provided by the `check_percentage/1` function. Each task is associated with a percentage representing its contribution to the overall project workload. The function uses `dets:select/2` to retrieve the percentages of all tasks belonging to a specific project and `lists:sum/1` to calculate their total.

The function returns both the calculated percentage and a Boolean value indicating whether the total is equal to 100. This allows the system to verify the distribution of the project workload across its associated tasks.

2.6 Client-server communication

Communication between clients and the central server is implemented through Erlang's message-passing mechanism. Clients send requests using the `!` operator, including the operation identifier, the required parameters, and the client process identifier, obtained through `self/0`. The server processes each request and sends a response to the originating client, which retrieves the result through a `receive` expression. Although message transmission is asynchronous, the implemented client interface follows a synchronous request-response pattern: after sending a request, the client waits for the corresponding result before continuing its execution. The implemented client request functions use a timeout mechanism set to 5000 milliseconds (5 seconds) to prevent indefinite blocking. If no matching response is received within this interval, the function returns the atom `timeout`. This mechanism prevents the client from waiting indefinitely for a response, although a timeout does not necessarily indicate that the requested operation has failed.

2.7 Error handling and data validation

The research management system implements a set of application-level validation mechanisms to ensure data consistency and prevent invalid operations on the DETS database.

Validation is performed by the server before inserting new records. When a request violates an implemented constraint, the corresponding function returns an error tuple in the form `{error, Reason}`, where `Reason` identifies the validation condition that was not satisfied.

Table 2: Validation scenarios and expected outcomes on the research project management system.

Test	**Description**	**Expected result**
Duplicate project ID	Prevents the insertion of a project with an existing identifier.	`project_already_exists`
Duplicate researcher ID	Prevents the insertion of a researcher with an existing identifier.	`researcher_already_exists`
Duplicate task ID	Prevents the insertion of a task with an existing identifier.	`task_already_exists`
Invalid project in participation	Rejects the association of a researcher with a non-existing project.	`project_not_found`
Invalid researcher in participation	Rejects the association of a non-existing researcher with a project.	`researcher_not_found`
Invalid project in task	Prevents task assignment to a non-existing project.	`project_not_found`
Invalid researcher in task	Prevents task assignment to a non-existing researcher.	`researcher_not_found`
Researcher not participating	Ensures that the assigned researcher participates in the corresponding project.	`researcher_not_in_project`

Test	Description	Expected result
Negative percentage	Rejects a task with a negative workload percentage.	`invalid_percentage`
Percentage overflow	Prevents the cumulative task workload from exceeding 100%.	`percentage_exceeds_100`
Existing project retrieval	Retrieves an existing project through client-server message exchange.	Project record
Missing project retrieval	Handles requests for non-existing projects.	`[]`
Valid project insertion	Verifies the insertion of a new project with a unique identifier.	`ok`
Valid researcher insertion	Verifies the insertion of a new researcher with a unique identifier.	`ok`
Valid participation insertion	Verifies the association of an existing researcher with an existing project.	`ok`
Valid task insertion	Accepts a task when all validation constraints are satisfied.	`ok`

Identifier uniqueness The `add_project/3`, `add_researcher/3`, and `add_task/6` functions verify whether the supplied identifier is already present in the corresponding DETS table.

Before inserting a new record, the system performs a lookup using `dets:lookup/2`. If a record with the same identifier already exists, the insertion is rejected.

Depending on the operation, the server returns one of the following errors:

- `project_already_exists`
- `researcher_already_exists`
- `task_already_exists`

These checks prevent new records from overwriting existing records with the same identifier.

Referential integrity The system verifies the existence of related records before creating associations between entities.

The `add_participation/2` function checks that both the specified project and researcher exist before inserting a participation record.

Similarly, the `add_task/6` function verifies that the referenced project and researcher are present in their respective tables.

If either entity is missing, the operation is rejected with one of the following errors:

- `project_not_found`
- `researcher_not_found`

These checks prevent the creation of associations or tasks referencing non-existing entities.

Researcher participation constraint An additional validation rule is implemented in the `add_task/6` function.

Before assigning a task to a researcher, the server verifies that the researcher is associated with the corresponding project.

The existing participation records are retrieved from the `partecipazioni` table, and membership is checked using `lists:member/2`.

If the association is not found, the server returns:

`{error,researcher_not_in_project}`

This constraint ensures that tasks can only be assigned to researchers participating in the corresponding project.

Workload percentage validation Each task is associated with a percentage representing its contribution to the overall workload of the corresponding project.

The `add_task/6` function verifies that the supplied percentage is numeric and falls within the interval from 0 to 100, inclusive.

Values outside this interval, as well as non-numeric values, are rejected with:

`{error,invalid_percentage}`

The system also enforces a cumulative workload constraint at the project level.

Before inserting a task, the server calls `check_percentage/1`, which retrieves the percentages of existing tasks belonging to the specified project using `dets:select/2`.

The percentages are summed using `lists:sum/1`.

The new task is accepted only if adding its percentage does not cause the cumulative project workload to exceed 100%.

Otherwise, the function returns:

`{error,percentage_exceeds_100}`

The `check_percentage/1` function also returns a Boolean indicator specifying whether the cumulative percentage is exactly equal to 100.

Importantly, a total below 100% is permitted during task insertion, allowing the workload to be allocated progressively.

Pattern matching and error propagation Validation logic relies on Erlang pattern matching to distinguish the outcomes of database operations.

For example, the system uses `case` expressions to distinguish an empty lookup result, indicating that a record does not exist, from a single-element list, indicating that the record is present.

The server also uses pattern matching within its `receive` expression to identify incoming requests based on their tuple structure and operation atoms.

Once a request has been processed, the resulting value, including any validation error, is returned to the requesting client through a message of the form:

`{result, Risultato}`

This approach provides explicit error reporting while keeping validation and database operations within the central server process.

2.8 Distributed system configuration

The research management system was initially developed using a single Erlang node, with client and server processes communicating through local message passing.

To explore Erlang's distributed programming capabilities, the application was subsequently tested using three independent Erlang runtime instances: one server node and two client nodes.

2.8.1 Node initialization and connectivity

The experiments were conducted using Erlang/OTP 29 on a Windows operating system. Three independent Erlang runtime instances were used to simulate a distributed client-server environment on a single physical machine.

The three nodes were started in separate terminal sessions using the following commands:

```
erl -sname server
erl -sname client1
erl -sname client2
```

The nodes were identified as:

```
server@DESKTOP-K09UC1B
client1@DESKTOP-K09UC1B
client2@DESKTOP-K09UC1B
```

All three nodes were executed on the same physical machine but operated as independent Erlang runtime instances.

Connectivity between each client node and the server node was checked using the `net_adm:ping/1` function.

```
net_adm:ping('server@DESKTOP-K09UC1B').
```

A response of `pong` indicated that communication with the server node had been established.

2.8.2 Server registration and remote requests

The central server process was started on the server node and registered under the name `research_server`.

This allowed client processes running on other Erlang nodes to address the server through its registered name and node identifier.

Requests were transmitted using Erlang's message-passing operator (`!`).

Each request was represented as a tuple containing the operation identifier, the required parameters, and the process identifier of the requesting client.

For example, researcher insertion requests followed the structure:

```
{add_researcher, Id, Nome, Ruolo, From}
```

The `From` parameter identified the requesting process and allowed the server to send the result back to the appropriate client.

2.8.3 Distributed Message Processing

The server used pattern matching within its `receive` expression to identify the requested operation according to the structure of the incoming message.

Once a request was received, the server executed the corresponding function, including the required database validation and insertion or retrieval operations.

The result was then returned to the requesting client through a message with the following structure:

```
{result, Risultato}
```

The client retrieved the response using a `receive` expression.

Although messages were transmitted asynchronously, the client-side interaction followed a request-response pattern.

All database operations were performed by the central server process, which managed the four DETS tables.

The client nodes did not maintain separate copies of the database.

This configuration was used to evaluate inter-node communication and the handling of requests originating from different client processes.

The corresponding experimental results are presented in the Results and Testing section.

2.9 Concurrent request testing procedure

To examine the handling of competing requests, two client processes running on separate Erlang nodes were configured to submit researcher insertion requests using the same identifier.

Both processes were prepared to send their requests upon receiving a start message. The server node then triggered the two clients in rapid succession.

This procedure was intended to examine how the central server handled competing operations involving the same database key.

3 Results and testing

The complete snapshot of the four DETS tables obtained during the final testing phase is provided in Appendix A.

3.1 Validation and functional testing

The implemented validation mechanisms were evaluated through the Erlang shell by submitting valid and invalid requests to the server process.

The tests aimed to verify the enforcement of application-level constraints, including project existence, workload percentage limits, and the rejection of invalid percentage values. Additional functional tests were performed to assess project retrieval operations.

Project insertion The correct insertion of a new project was verified through the Erlang shell. Before submitting the insertion request, the absence of the project with identifier 101 was confirmed.

```
research:request_get_project(ServerPid4, 101).
```

Observed response:

```
[]
```

A new project was then submitted to the server:

```
ServerPid4 ! {add_project, 101,
    "Studio melanoma", clinico, self()}.
```

The server response was retrieved using:

```
receive
    {result, Risposta101} -> Risposta101
after 5000 ->
    timeout
end.
```

Observed response:

```
ok
```

To verify that the project had been successfully stored in the DETS database, a retrieval request was subsequently performed:

```
research:request_get_project(ServerPid4, 101).
```

Observed response:

```
[{101,"Studio melanoma",clinico}]
```

The successful insertion and subsequent retrieval confirmed that the server correctly processed the project creation request and that the corresponding record was stored in the DETS database.

Duplicate researcher insertion The system was tested to verify the enforcement of unique researcher identifiers.

Two insertion requests were submitted using the same researcher identifier (ID = 9700) but different personal information.

The first request inserted a researcher identified as "Marco Verdi", with the role of biologist.

The second request attempted to insert a different researcher, "Laura Rossi", with the role of physician, using the same identifier.

The first insertion was successfully completed, whereas the second request was rejected.

Observed response to the duplicate insertion:

```
{error,researcher_already_exists}
```

A subsequent database lookup confirmed that the original researcher record was preserved:

```
[{9700,"Marco Verdi",biologo}]
```

These results demonstrated that the server correctly enforced researcher identifier uniqueness and prevented existing records from being overwritten by duplicate insertion requests.

Validation of workload percentage limits A task insertion request was submitted with a workload percentage that would exceed the maximum allowed total of 100% for the corresponding project.

```
research:request_add_task(
    ServerPid, 9201, 1,
    "Data analysis", 9001, todo, 25).
```

Observed response:

```
{error,percentage_exceeds_100}
```

The server rejected the insertion, demonstrating the enforcement of the maximum workload allocation constraint at the project level.

Validation of negative percentages The system was tested by submitting a task insertion request with a negative workload percentage.

```
research:request_add_task(
    ServerPid, 9203, 2,
    "Preliminary analysis", 101, todo, -10).
```

Observed response:

```
{error,invalid_percentage}
```

The request was rejected, confirming that negative workload percentages are not accepted.

Project retrieval The retrieval functionality was tested using both existing and non-existing project identifiers.

For an existing project:

```
research:request_get_project(ServerPid, 1).
```

Observed response:

```
[{1,"Studio melanoma",clinico}]
```

For a non-existing project:

```
research:request_get_project(ServerPid, 9999).
```

Observed response:

```
[]
```

These results confirmed that the system correctly retrieved existing project records and returned an empty list when the requested identifier was not present in the database.

Overall, the functional tests confirmed that the implemented validation mechanisms correctly rejected invalid task insertion requests, including negative workload percentages and allocations exceeding the maximum allowed project workload.

These results provide evidence that the tested application-level constraints were correctly enforced by the server.

3.2 Distributed communication testing

Distributed communication was evaluated using three Erlang nodes running on the same host machine: one server node and two client nodes.

The nodes were started independently using the following commands:

```
erl -sname server
erl -sname client1
erl -sname client2
```

The resulting node identifiers were:

```
server@DESKTOP-K09UC1B
client1@DESKTOP-K09UC1B
client2@DESKTOP-K09UC1B
```

Node connectivity The first test aimed to verify network connectivity between the client nodes and the server node.

From each client node, the following command was executed:

```
net_adm:ping('server@DESKTOP-K09UC1B').
```

Observed response:

```
pong
```

The successful responses confirmed that both client nodes could establish communication with the server node through Erlang distribution.

Remote client-server communication After verifying node connectivity, communication between distributed processes was tested.

The server process was registered under the name `research_server`, allowing remote clients to address it through its registered name and node identifier.

A client request was submitted to the remote server using the following message structure:

```
{add_researcher, Id, Nome, Ruolo, From}
```

The server identified the requested operation through pattern matching within its receive expression, executed the corresponding function, and returned the result to the requesting process.

During testing, a researcher with identifier 9501 was successfully inserted through a remote client request.

The successful operation confirmed that a request originating from a client node could be processed by the server node and that the corresponding response could be returned to the requesting client.

Distributed communication outcome The tests demonstrated that the implemented client-server architecture supports communication between processes running on different Erlang nodes.

Both client nodes successfully established connectivity with the server, and remote requests were processed through Erlang message passing.

Although the nodes were executed on the same physical machine, they operated as independent Erlang runtime instances, providing a controlled environment for evaluating distributed communication.

3.3 Concurrent request handling

The system was tested to evaluate how the server handles competing insertion requests originating from different Erlang client nodes.

Two client processes, running on separate Erlang nodes, were configured to submit requests for the insertion of a researcher with the same identifier (ID = 10000).

The first client attempted to insert the following researcher:

```
{10000,"Mario Bianchi",biologo}
```

The second client attempted to insert:

```
{10000,"Laura Verdi",medico}
```

Both client processes were prepared to send their requests upon receiving a start message. The server node then triggered both clients in rapid succession.

The observed responses were:

```
Client 1:
ok

Client 2:
{error,researcher_already_exists}
```

A subsequent lookup of the researcher record was performed:

```
dets:lookup(ricercatori, 10000).
```

Observed response:

```
[{10000,"Mario Bianchi",biologo}]
```

These results demonstrated that the server successfully processed one insertion request while rejecting the competing request using the same identifier.

This behavior is consistent with the sequential processing of messages within the server receive loop, where each request is handled before the next one is processed.

The test also confirmed that the duplicate identifier validation prevented the second request from overwriting the existing record.

3.4 Data persistence testing

Data persistence was evaluated to verify that records stored in the DETS database remained available after the server process was stopped and subsequently restarted.

The test was performed using a researcher record with identifier 9501, previously inserted through a remote client request.

Server restart and data retrieval After the researcher had been successfully inserted, the server process was stopped, closing the DETS tables.

The server was subsequently restarted, reopening the existing DETS files.

A lookup operation was then performed to verify whether the previously inserted researcher record was still available.

The retrieval confirmed that the researcher with identifier 9501 remained stored in the database after the server restart.

Persistence test outcome The successful retrieval demonstrated that the implemented DETS-based storage mechanism preserved the tested record across a controlled server shutdown and restart.

Unlike in-memory ETS tables, DETS tables store records in files, allowing previously inserted data to be recovered when the corresponding files are reopened.

4 Conclusion

Although the implemented system was developed and tested on a limited scale, the project provided an opportunity to explore the main concepts of Erlang programming within a concrete application. Possible future developments could include testing the system across physically separate machines, evaluating its behavior with a larger number of concurrent clients, and extending the implemented functionalities and error-handling mechanisms.

References

[1] Erlang/OTP. Erlang Reference Manual. https://www.erlang.org/doc/system/reference_manual.html. Accessed: 12 October 2026.

A Supplementary Material: DETS Database Records

This appendix reports a snapshot of the records stored in the four DETS tables during the final testing phase.

The records were retrieved directly from the Erlang server node using the `dets:match_object/2` function, without modifying the database contents.

The data are presented in their original Erlang tuple format to document the structure and contents of the database used during testing.

A.1 Projects

The `progetti` table contains four records.

```
[{1,"Studio melanoma",clinico},
 {2,"Studio polmone",clinico},
 {5,"Studio colon",biologico},
 {101,"Studio melanoma",clinico}]
```

A.2 Researchers

The `ricercatori` table contains eleven records.

```
[{10,"Mario",principal_investigator},
 {11,"Anna",data_manager},
 {12,"Luca",data_analyst},
 {101,"Mario Rossi",biologo},
 {9001,"Luca Bianchi",researcher},
 {9501,"Giulia Verdi",biologo},
 {9600,"Luca Neri",medico},
 {9700,"Marco Verdi",biologo},
 {9800,"Paolo Rossi",biologo},
 {9900,"Laura Verdi",medico},
 {10000,"Mario Bianchi",biologo}]
```

A.3 Participations

The `partecipazioni` table contains five records.

```
[{1,10},
 {1,11},
 {1,12},
 {1,9001},
 {2,101}]
```

A.4 Tasks

The `tasks` table contains six records.

```
[{101,1,"Redazione protocollo",10,completed,30},
 {102,1,"Raccolta dati",11,running,30},
 {103,1,"Analisi dei dati",12,todo,20},
 {104,1,"Gestione amministrativa",10,todo,20},
 {1003,2,"Analisi biomarcatori",101,todo,25},
 {9302,2,"Revisione dei risultati",101,todo,10}]
```